

8 - AJAX (API REST)

Como ya vimos anteriormente, las API (Application Programming Interfaces) son conjuntos de reglas y protocolos que permiten que diferentes aplicaciones o sistemas se comuniquen y compartan datos o funcionalidades. En el contexto de la arquitectura cliente-servidor web, las API desempeñan un papel fundamental al facilitar la comunicación entre el cliente y el servidor.

Introducción a REST

REST (Representational State Transfer) es un estilo de arquitectura para sistemas distribuidos, comúnmente utilizado en aplicaciones web. Define un conjunto de restricciones y principios para interactuar con recursos a través del protocolo HTTP. En un sistema RESTful, los recursos se representan mediante URLs y se manipulan utilizando operaciones estándar.

¿Cómo funciona REST?

1. Cliente-Servidor

REST se basa en la separación de responsabilidades entre el cliente (que consume recursos) y el servidor (que los ofrece). Esto facilita la escalabilidad y el mantenimiento.

2. Recursos identificados mediante URLs

Cada recurso tiene una dirección única (URL) que permite acceder a él, como `/users` o `/users/123`.

3. Uso de verbos HTTP

Las operaciones realizadas sobre los recursos utilizan métodos HTTP estándar

4. Formato estándar para intercambio de datos

REST utiliza formatos como JSON o XML para transferir datos. JSON es el más común debido a su simplicidad y compatibilidad con JavaScript.

5. Sin estado (Stateless)

Cada petición HTTP es independiente, lo que significa que el servidor no guarda información sobre el estado del cliente entre peticiones. Toda la información necesaria para procesar una solicitud debe ser enviada con ella.

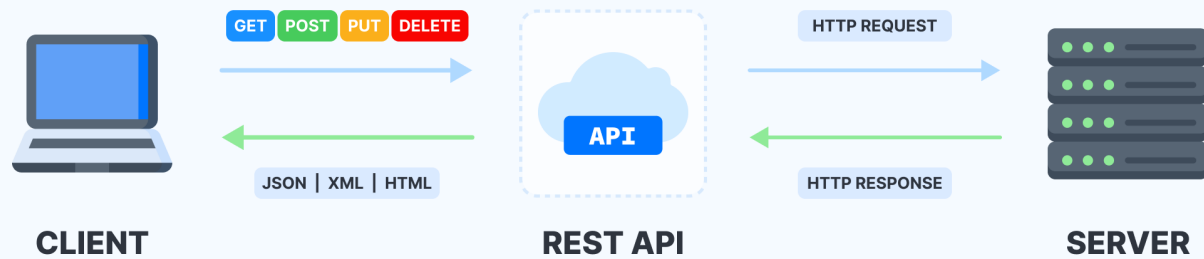
Principales Verbos HTTP

Verbo	Propósito	Ejemplo
GET	Obtener un recurso o lista de recursos	<code>GET /users</code> (lista de usuarios)
POST	Crear un nuevo recurso	<code>POST /users</code> (crear un usuario)
PUT	Actualizar un recurso existente	<code>PUT /users/123</code> (actualizar usuario con ID 123)
DELETE	Eliminar un recurso	<code>DELETE /users/123</code> (eliminar usuario con ID 123)

Diagrama ilustrativo del flujo REST

El siguiente diagrama muestra cómo interactúan un cliente y un servidor RESTful mediante solicitudes HTTP:

REST API Model



1. El cliente envía una solicitud HTTP con un verbo (GET, POST, etc.) a un recurso identificado por su URL.
2. El servidor responde con un estado HTTP (e.g., `200 OK`, `404 Not Found`) y, si es necesario, un cuerpo con datos en formato JSON o XML.

Consumir una API

Vamos a hacer una web de citas y para ello vamos a necesitar datos de nuestros usuarios. En este caso vamos a utilizar una API que nos suministra usuarios aleatorios falsos

Si nos leemos [la documentación](#) veremos que tenemos que hacer una llamada a una url concreta: <https://randomuser.me/api/>, para que nos devuelva un usuario aleatorio.

Si hacemos una petición a esta URL desde un cliente REST (como Postman, Thunder, RapidApi, etc..) podemos ver la respuesta que obtenemos:

```
JSON
{
  "results": [
    {
      "gender": "female",
      "name": {
        "title": "Miss",
        "first": "Jessica",
        "last": "Long"
      },
      "location": {
        "street": {
          "number": 7807,
          "name": "School Lane"
        },
        "city": "Nottingham",
        "state": "Derbyshire",
        "country": "United Kingdom",
        "postcode": "J64 3WD",
        "coordinates": {
          "latitude": "-7.4296",
          "longitude": "75.8710"
        },
        "timezone": {
          "offset": "+11:00",
          "description": "Magadan, Solomon Islands, New Caledonia"
        }
      },
      "email": "jessica.long@example.com",
      "login": {
```

```

        "uuid": "782fcb94-0aa9-437c-9fdd-45c1a1bc73a1",
        "username": "ticklishelephant382",
        "password": "sonny",
        "salt": "NOJLBVL9",
        "md5": "ff8f23a652c50ea1649d083b6c371483",
        "sha1": "7390fe99362d16d4330307b7c4c7b12c1f50d394",
        "sha256": "269f412d4b2143077134336d3d45fad0c4417d4db0e0ff08191e0e2d38efbb9a"
    },
    "dob": {
        "date": "1989-11-11T16:04:35.958Z",
        "age": 34
    },
    "registered": {
        "date": "2004-01-09T05:13:15.953Z",
        "age": 20
    },
    "phone": "016977 48651",
    "cell": "07844 454474",
    "id": {
        "name": "NINO",
        "value": "PL 17 67 54 W"
    },
    "picture": {
        "large": "https://randomuser.me/api/portraits/women/60.jpg",
        "medium": "https://randomuser.me/api/portraits/med/women/60.jpg",
        "thumbnail": "https://randomuser.me/api/portraits/thumb/women/60.jpg"
    },
    "nat": "GB"
}
],
"info": {
    "seed": "bfa0d4431e1e055b",
    "results": 1,
    "page": 1,
    "version": "1.4"
}
}

```

Diferentes URLS (endpoints) pueden aceptar diferentes parámetros y con eso podremos solicitar específicamente lo que queremos dentro de las opciones que nos ofrece esta API.

Por ejemplo si llamamos a <https://randomuser.me/api/?gender=female> nos devuelve una respuesta con un usuario aleatorio de género femenino.

JSON

Si nos fijamos en lo que nos devuelve la API, vemos que a primera vista parece un objeto de JavaScript, pero en realidad es un formato JSON.

JSON, cuyas siglas significan “JavaScript Object Notation” - “Notación de Objeto de JavaScript” es un formato de texto que nos permite convertir JavaScript en texto para el intercambio de datos.

Podemos convertir JavaScript en JSON y viceversa. Cuando tratemos datos que nos llegan desde una petición, estos nos llegarán en formato texto (JSON) y deberemos convertirlo en JavaScript para poder manejar y utilizar estos datos.

Siempre que enviemos o recibamos información vamos a tener que trabajar con objetos. Pero para poder convertirlos desde y en formato JSON vamos a necesitar el objeto JSON de JavaScript.

Pongamos que queremos enviar la información de un objeto que hemos creado:

```

let user = {
    nick: 'SuperDad',
    age: 36,

```

```
name: "Jack",
surname: "Sparrow",
city: "Lyon",
country: "France"
}
```

JSON.stringify()

A través de la función `JSON.stringify()`, podemos pasar un objeto javascript a JSON, simplemente tenemos que pasarle por parámetro el objeto que queremos convertir.

```
JSON.stringify(user)
```

```
'{"nick":"SuperDad","age":36,"name":"Jack","surname":"Sparrow","city":"Lyon","country":"France"}
```

JSON.parse()

```
JSON.parse('{"nombre":"Thor","especie":"Perro","raza":"Gran Danés","edad":8}')
```

```
{
  nombre: 'Thor',
  especie: 'Perro',
  raza: 'Gran Danés',
  edad: 8
}
```

Note

Lo convertimos en texto para enviarlo. Cuando pedimos información a una página de internet esta información nos llega en formato texto y queremos parsearlo a Objeto javascript para poder manejarlo e incluirlo en nuestra página.

AJAX

JavaScript puede enviar peticiones de red al servidor y cargar nueva información siempre que se necesite.

Por ejemplo, podemos utilizar una petición de red para:

- Crear una orden
- Cargar información de usuario
- Recibir las últimas actualizaciones desde un servidor

Se utiliza el término global “AJAX” (abreviado Asynchronous JavaScript And XML, en español: “JavaScript Asíncrono y XML”) para referirse a las peticiones de red originadas desde JavaScript. Sin embargo, no estamos necesariamente condicionados a utilizar el formato XML dado que el término es antiguo y es por esto que el acrónimo XML se encuentra aquí.

¿Cómo funciona AJAX en Fetch?

Fetch es una API nativa de JavaScript que simplifica las solicitudes de red, reemplazando métodos más antiguos como `XMLHttpRequest`. Te permite hacer llamadas HTTP (GET, POST, etc.) de forma asíncrona, y se basa en **promesas**, lo que facilita trabajar con respuestas y errores.

- Más simple y moderno que `XMLHttpRequest`.
- Basado en promesas, lo que lo hace más legible y manejable.
- Compatible con operaciones CRUD (Create, Read, Update, Delete) a través de HTTP.

Veamos como utilizarlo:

```
fetch('url', {options})
```

- `url`: representa la URL a la que deseamos pedir
- `options`: representa el objeto de configuración. que nos permite definir parámetros como puede ser un método, los encabezados de nuestra petición, etc.

Info

Si no especificamos ninguna configuración con el objeto `options`, se ejecutará una simple petición GET.

El navegador lanzará la petición de inmediato y el método `fetch()` devolverá una promesa que se resolverá cuando la petición haya sido completada y tengamos una respuesta. A partir de ahí tendremos que procesarla.

Para eso utilizaremos el método `then()` que recibirá en su callback el cuerpo de la respuesta como primer argumento.

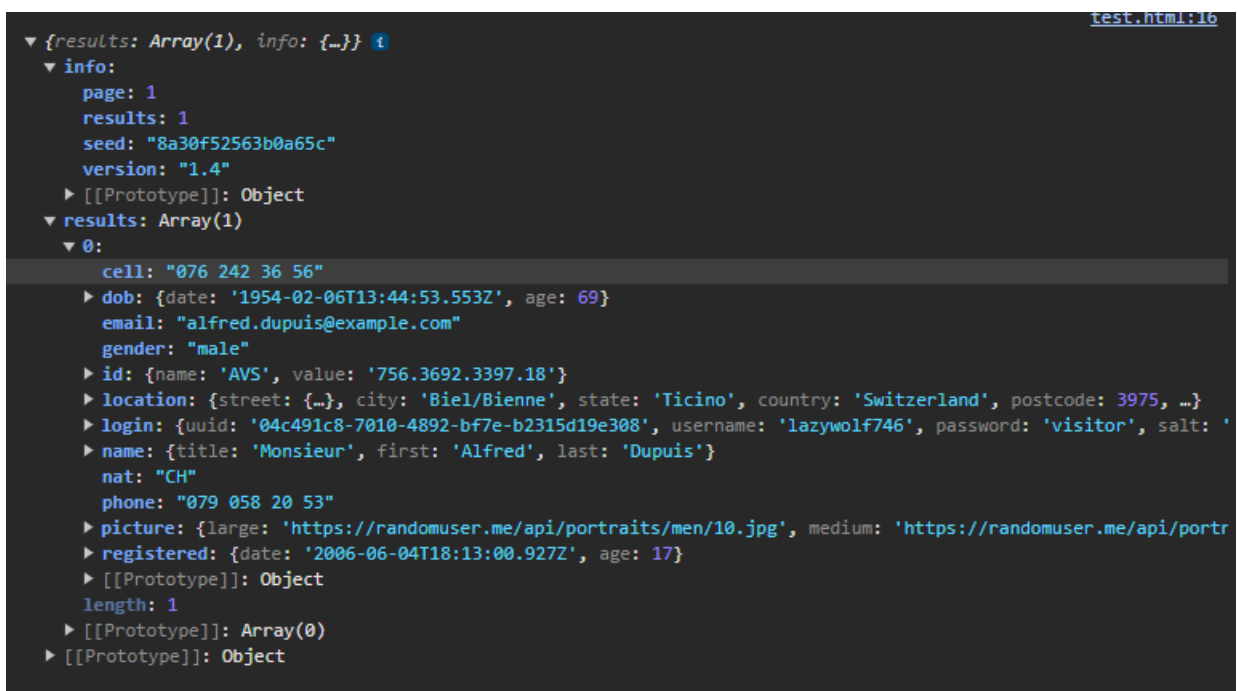
La respuesta en este caso nos llegará en formato JSON y podemos utilizar el método `.json()` para convertir el cuerpo de la respuesta de nuestra llamada de json a un objeto JavaScript con el método;

```
fetch('https://randomuser.me/api/')
.then(function recogerRespuesta(respuesta){
  return respuesta.json()
})
```

Después utilizaremos otro `.then()` para que cuando la promesa que devuelve el método `json()` se resuelva, podamos procesar el objeto resultante. En este caso, lo que pasamos como argumento en el callback sería el objeto JavaScript ya parseado y convertido del JSON de la respuesta.

```
fetch('https://randomuser.me/api/')
  .then(function recogerRespuesta(respuesta) {
    return respuesta.json()
  })
  .then(function imprimirObjeto(datos){
    console.log(datos)
  })
```

Si mostramos por consola el valor de 'datos', el parámetro que recibe la función `imprimirObjeto()`, vemos cómo lo que recibimos es un objeto JavaScript ya convertido:



Ahora si queremos mostrar el nombre del usuario que nos devuelve, tendremos que acceder al objeto 'datos' a la propiedad 'results' que es un array, en este a su primer índice (0), después a name, y por último acceder a la propiedad 'first':

```
fetch('https://randomuser.me/api/')
  .then(function recogerRespuesta(respuesta) {
    return respuesta.json()
  })
  .then(function imprimirObjeto(datos) {
    console.log(datos)
    console.log(datos.results[0].name.first)
  })
```

También podemos introducir este nombre como contenido de un párrafo para mostrarlo en el html

```
fetch('https://randomuser.me/api/')
  .then(function recogerRespuesta(respuesta) {
    return respuesta.json()
  })
  .then(function imprimirObjeto(datos) {
    console.log(datos)
    console.log(datos.results[0].name.first)
    document.getElementById('nombre').innerHTML = `<p>${datos.results[0].name.first} ${datos
  })
```